

LECTURE 10

CPU Scheduling: Concepts and FCFS

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Why CPU scheduling exists: maximizing CPU utilization by always having something ready to run
- The CPU burst / I/O burst cycle, and how its typical shape drives scheduling theory
- The CPU scheduler vs. the dispatcher — selecting a process vs. actually switching to it
- Preemptive vs. non-preemptive scheduling
- The five standard scheduling criteria: utilization, throughput, turnaround time, waiting time, response time
- First-Come, First-Served (FCFS) scheduling, worked through a full numeric example — and the convoy effect it exposes

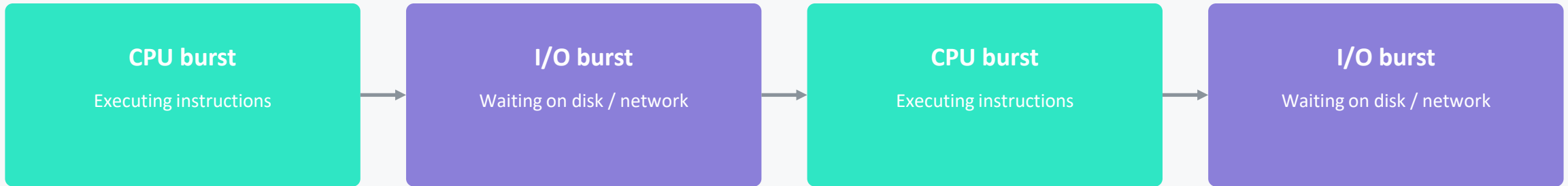
Why CPU Scheduling Exists

A single CPU can run exactly one process's instructions at any given instant. Multiprogramming keeps several processes in memory simultaneously so that when the running process needs to wait — almost always for I/O — the OS can immediately switch the CPU to a different, ready process instead of leaving it idle.

CPU scheduling is the policy layer that decides, among every process sitting in the ready queue, which one gets the CPU next.

Done well, the CPU is almost never idle while there's ready work sitting in the queue; done poorly, the CPU sits idle constantly even though the system is full of processes that would happily use it.

The CPU Burst / I/O Burst Cycle



A process alternates between these two kinds of bursts until it terminates. The distribution of CPU burst lengths is heavily skewed: a huge number of short bursts, and a small number of much longer ones — this skew is exactly why Lecture 11's Shortest-Job-First scheduling pays off.

I/O-bound vs. CPU-bound processes

A process dominated by many short CPU bursts separated by I/O waits is I/O-bound; a process dominated by long CPU bursts with only occasional I/O is CPU-bound. Favor CPU-bound processes and I/O-bound ones feel sluggish; favor I/O-bound processes and CPU-bound ones never finish.

The CPU Scheduler and the Dispatcher

CPU Scheduler

The decision-maker (also the short-term scheduler). Looks at the ready queue and selects which process runs next, according to whatever scheduling algorithm is in effect.

Dispatcher

The mechanism. Once the scheduler has decided, performs the context switch: saving the old process's state, loading the new process's state, switching the CPU to user mode at the new process's next instruction.

Dispatch latency

The time the dispatcher spends performing a context switch — pure bookkeeping overhead that does no useful work for either process. Every context switch pays this cost, which is why Round Robin's quantum size (Lecture 11) treats too many context switches as a real performance problem.

Preemptive vs. Non-Preemptive Scheduling

Non-Preemptive

Once a process is given the CPU, it keeps it until it voluntarily gives it up (blocks on I/O) or terminates. The scheduler can only make a new decision at those moments.

Preemptive

The OS can forcibly take the CPU away from a running process before it finishes or blocks — e.g. a higher-priority process became ready, or a time slice expired.

Preemption has a real cost

A process can be preempted mid-update of data shared with another process. If left half-updated, the next process to touch that data can corrupt it — extremely hard to reproduce. Preemptive scheduling is what makes synchronization (starting Lecture 13) necessary in the first place.

Scheduling Criteria

- **CPU utilization** — percentage of time the CPU is doing useful work rather than sitting idle. Higher is better.
- **Throughput** — the number of processes completed per unit of time. Higher is better.
- **Turnaround time (TAT)** — total time from arrival to completion. $TAT = \text{completion time} - \text{arrival time}$.
- **Waiting time** — total time spent in the ready queue, not running and not doing I/O. For single-burst examples: $\text{waiting time} = \text{turnaround time} - \text{burst time}$.
- **Response time** — time from arrival to the process's first output — not when it finishes, just when it starts visibly responding.

Waiting time and turnaround time are not the same number

A process with a long CPU burst can have a short waiting time (if it starts immediately) and still a long turnaround time (because the burst itself takes a while).

First-Come, First-Served (FCFS)

Whichever process arrives in the ready queue first gets the CPU first, and keeps it until it finishes (FCFS is non-preemptive). Implemented with nothing more than a plain FIFO queue.

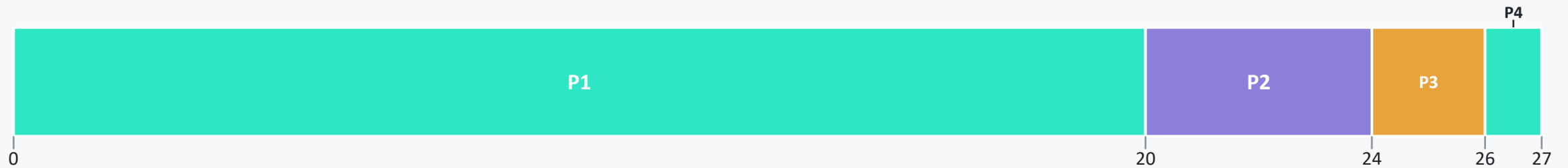
Worked Example: four processes arrive as follows.

Process	Arrival Time	Burst Time
P1	0	20
P2	1	4
P3	2	2
P4	3	1

All four processes are ready well before P1 finishes, so FCFS simply runs them in arrival order: P1, then P2, then P3, then P4.

Gantt Chart: FCFS Schedule

Arrival order P1, P2, P3, P4



- **P1** starts the instant it arrives at $t=0$, runs its full burst of 20, finishing at $t=20$
- **P2** arrived at $t=1$ but the CPU is busy until $t=20$; starts at $t=20$, runs 4, finishes at $t=24$
- **P3** arrived at $t=2$, starts at $t=24$, runs 2, finishes at $t=26$
- **P4** arrived at $t=3$, starts at $t=26$, runs 1, finishes at $t=27$

FCFS: Waiting and Turnaround Times

Process	Arrival	Burst	Start	Completion	Waiting	Turnaround
P1	0	20	0	20	$0-0=0$	$20-0=20$
P2	1	4	20	24	$20-1=19$	$24-1=23$
P3	2	2	24	26	$24-2=22$	$26-2=24$
P4	3	1	26	27	$26-3=23$	$27-3=24$

Average waiting time = $(0 + 19 + 22 + 23) / 4 = 64 / 4 = 16$ Average turnaround time = $(20 + 23 + 24 + 24) / 4 = 91 / 4 = 22.75$

Notice how badly P2, P3, and P4 fare — each has a tiny burst, yet each waits roughly twenty units purely because they arrived just after a long process. This is the convoy effect: one long CPU-bound process holds the CPU, and a whole convoy of short processes piles up behind it — much like a slow truck on a single-lane road backing up every car behind it.

The Convoy Effect, Made Concrete

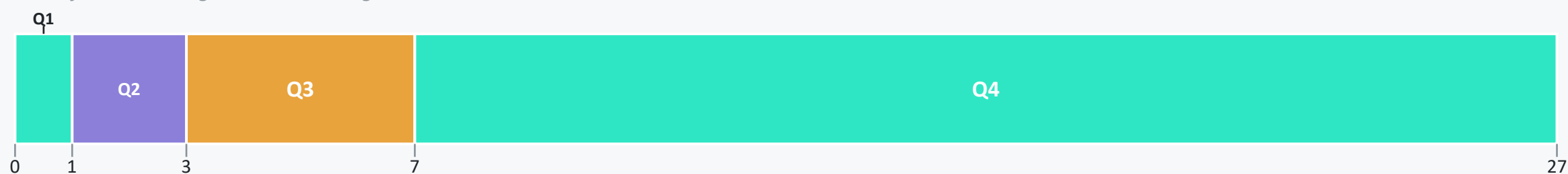
Run the exact same four burst lengths through FCFS again, but suppose they arrived in the opposite order: shortest burst first, longest last.

Process	Arrival Time	Burst Time
Q1	0	1
Q2	1	2
Q3	2	4
Q4	3	20

Q1 starts at $t=0$, runs 1, finishes at $t=1$. Q2 starts at $t=1$, runs 2, finishes at $t=3$. Q3 starts at $t=3$, runs 4, finishes at $t=7$. Q4 starts at $t=7$, runs 20, finishes at $t=27$.

Gantt Chart: Same Burst Lengths, Reversed Order

Same four burst lengths, short-to-long arrival order



Process	Arrival	Burst	Start	Completion	Waiting	Turnaround
Q1	0	1	0	1	0	1
Q2	1	2	1	3	0	2
Q3	2	4	3	7	1	5
Q4	3	20	7	27	4	24

Same Work, Wildly Different Averages

Average waiting time = $(0 + 0 + 1 + 4) / 4 = 5 / 4 = 1.25$

Same four burst lengths (1, 2, 4, 20 in both scenarios), same total work, same final completion time ($t=27$ either way) — and yet average waiting time fell from 16 to 1.25 purely by changing which process happened to arrive first.

FCFS's performance is entirely at the mercy of arrival order, and it has no mechanism to notice that a newly-arrived process is short and let it cut ahead of a long one already running — because FCFS is non-preemptive, nothing can interrupt the long process once it starts.

FCFS's one genuine advantage

Despite the convoy effect, FCFS is never unfair in a different sense: no process can be starved forever, because the queue only ever moves forward. Every later algorithm in this unit trades away some of that simplicity to fix the convoy effect.

Key Takeaways

- CPU scheduling exists to maximize CPU utilization in a multiprogrammed system — always having something ready to run the instant the CPU frees up
- Processes alternate between CPU bursts and I/O bursts; burst lengths skew toward many short bursts and few long ones, which later motivates Shortest-Job-First
- The CPU scheduler decides which process runs next; the dispatcher performs the actual context switch, at a real cost called dispatch latency
- Non-preemptive scheduling never interrupts a running process; preemptive scheduling can — at the cost of needing synchronization
- FCFS is simple and starvation-free, but entirely at the mercy of arrival order, which produces the convoy effect — the worked example's swing from average waiting time 16 down to 1.25, using the exact same burst lengths, demonstrates it